

Actividad I: Microcontroladores

Selección y Generación de Frecuencias con ATmega328P

Ingeniería – FES Cuautitlán

27 de septiembre de 2026

1 Resumen

El presente reporte documenta el diseño e implementación de un algoritmo en lenguaje ensamblador para el microcontrolador ATmega328P de la tarjeta de desarrollo Arduino UNO. El objetivo principal fue programar dos pines como entradas para seleccionar, mediante combinaciones lógicas, una de cuatro frecuencias de salida (100 kHz, 500 kHz, 1 MHz y 2 MHz) a través del pin conectado al LED integrado (PB5). Se analizan las limitaciones de la arquitectura y la relación entre los ciclos de instrucción y la frecuencia máxima alcanzable por el hardware, validando el comportamiento teórico con los resultados obtenidos en simulación.

2 Introducción

La generación de señales cuadradas a frecuencias específicas es una tarea fundamental en el diseño de sistemas embebidos, aplicable en modulación por ancho de pulsos, relojes de sincronización y protocolos de comunicación. Al utilizar el lenguaje ensamblador, se obtiene un control determinista sobre cada ciclo del procesador. En esta actividad, se evalúa la capacidad de respuesta de la arquitectura AVR operando con un cristal de 16 MHz, enfrentando el desafío de equilibrar la lectura constante de las entradas (polling) con la temporización estricta necesaria para generar señales de hasta 2 MHz sin utilizar interrupciones de hardware ni temporizadores (timers) por hardware.

3 Funcionamiento del Sistema

3.1 Microprocesador

El sistema se basa en el microcontrolador ATmega328P, el cual posee una arquitectura RISC (Reduced Instruction Set Computer) avanzada tipo AVR de 8 bits. La mayoría de sus instrucciones se ejecutan en un solo ciclo de reloj gracias al uso de *pipelining*, lo que le permite alcanzar rendimientos cercanos a 1 MIPS por MHz. Para este algoritmo, se utilizaron los registros de propósito general (`r16` y `r17`) para el manejo de datos y contadores de retardo. El *Stack Pointer* (SP) fue inicializado al final de la memoria RAM, un paso indispensable para asegurar la estabilidad del procesador.

3.2 Memorias

El ATmega328P cuenta con tres tipos principales de memoria:

- **SRAM (2 KB):** Memoria volátil utilizada para almacenar la pila (Stack) y variables dinámicas. Aunque en este diseño se evitaron las subrutinas de retardo para maximizar la velocidad, la pila requiere inicialización.

- **FLASH (32 KB):** Memoria no volátil donde se graba el algoritmo generado (.hex) tras ser ensamblado en Microchip Studio.
- **EEPROM (1 KB):** Memoria de datos no volátil. No fue requerida para el funcionamiento de esta práctica.

3.3 Interfaz para Periféricos

La interacción con el exterior se manejó a través de los puertos I/O, específicamente el Puerto B.

- **Registro DDRB:** Se configuró el bit 5 como salida (1) y los bits 0 y 1 como entradas (0).
- **Registro PORTB:** Se utilizó para activar las resistencias *pull-up* internas en los pines de entrada (PB0 y PB1), garantizando estados lógicos definidos en ausencia de señal, y para alternar los estados lógicos de salida en PB5.
- **Registro PINB:** Utilizado para leer el estado físico actual de los pines en cada ciclo iterativo.

4 Algoritmo del Programa

El algoritmo sigue una estructura de ciclo infinito (polling). Primero, lee el estado de PINB y aplica una máscara lógica (AND) para aislar los bits 0 y 1. Mediante instrucciones de comparación (cpi) y salto condicional (breq), redirige el flujo hacia la rutina de frecuencia correspondiente.

Para lograr la precisión, se eliminaron las subrutinas rcall (que consumen ciclos adicionales) y se implementaron retardos en línea. Las frecuencias bajas (100 kHz) utilizan bucles decrecientes, mientras que las frecuencias altas (500 kHz y 1 MHz) utilizan la instrucción nula nop (1 ciclo de reloj) para rellenar los tiempos de espera exactos.

5 Análisis de los Resultados

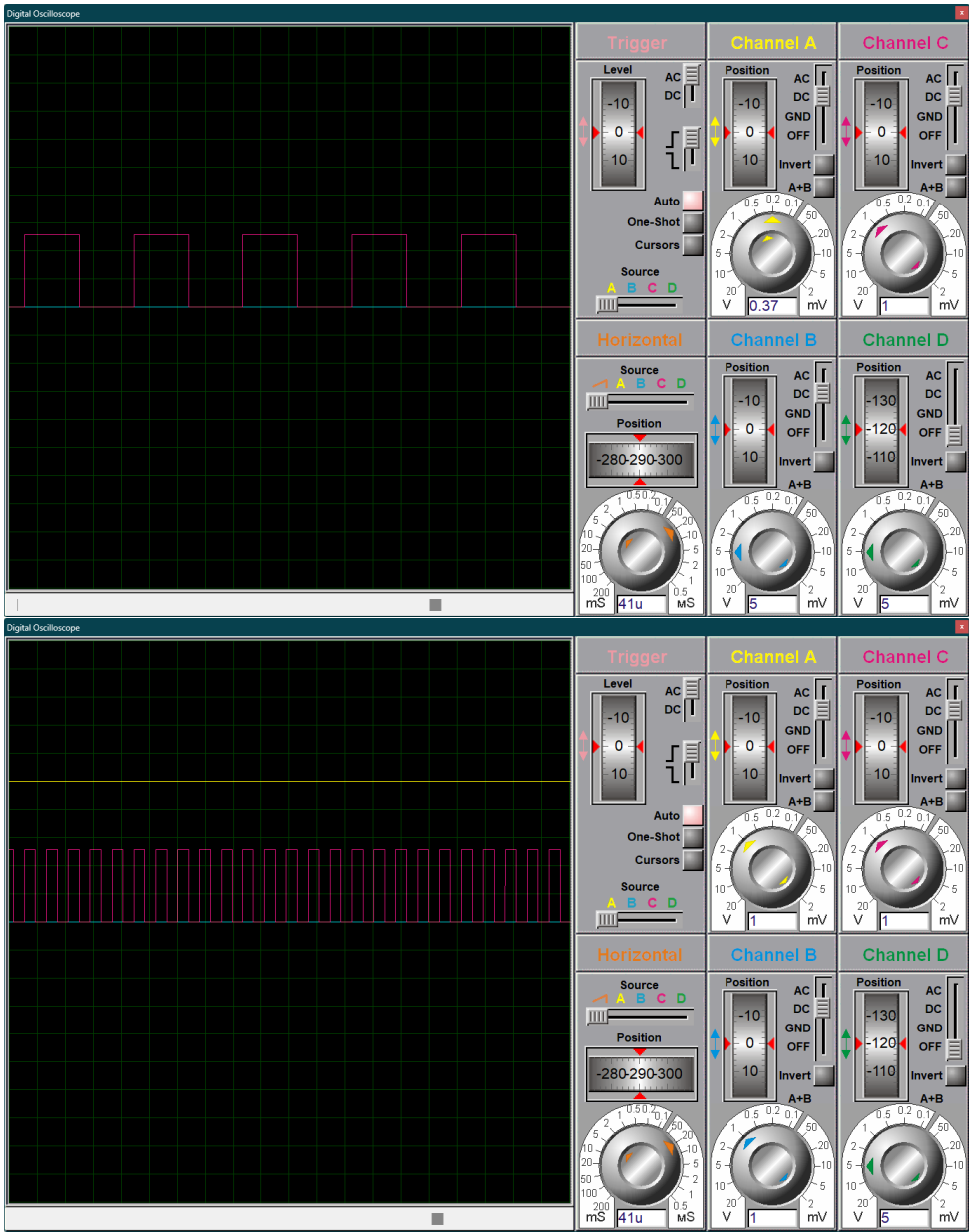
Considerando un reloj principal de 16 MHz, el número de ciclos requeridos por periodo (T) se rige por la ecuación:

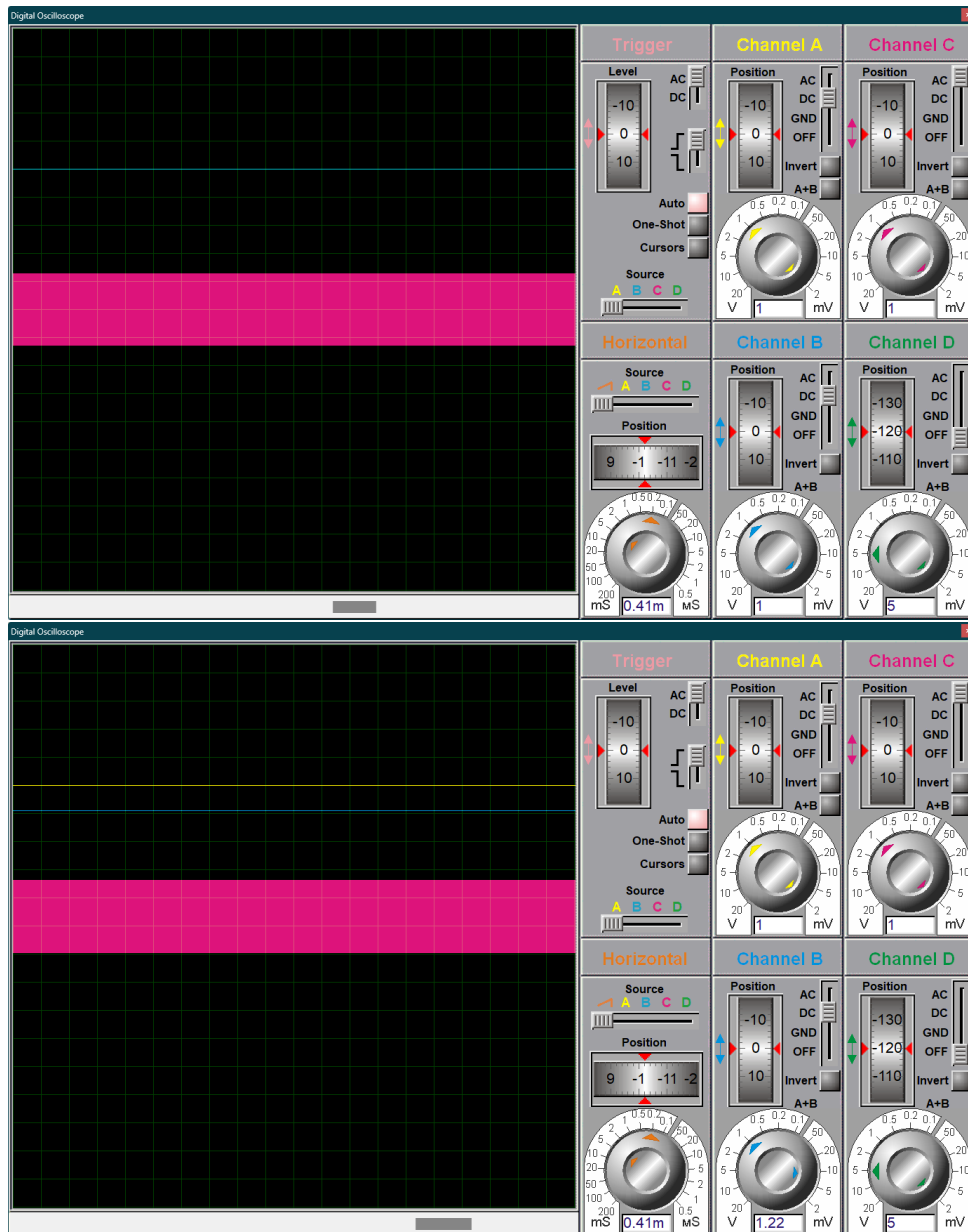
$$Ciclos = \frac{16,000,000}{f_{deseada}} \quad (1)$$

A continuación, se describen los resultados obtenidos tras simular el código en el osciloscopio virtual:

- **Entrada 00 (100 kHz):** Se requieren 160 ciclos por periodo (80 en estado alto y 80 en bajo). La simulación muestra pulsos anchos y simétricos (Ciclo de trabajo de 50%).
- **Entrada 01 (500 kHz):** Se requieren 32 ciclos (16 en estado alto, 16 en bajo). Se compensaron los ciclos de la rutina de lectura. La onda se visualiza densa pero claramente definida en la simulación.
- **Entrada 10 (1 MHz):** Se requieren 16 ciclos. Fue necesario utilizar 6 instrucciones nop en estado alto, logrando que el ciclo cuadre exactamente con los 8 ciclos que toma la validación lógica en el estado bajo.
- **Entrada 11 (2 MHz - Límite de Hardware):** En la teoría matemática requeriría 8 ciclos por periodo (4 en alto, 4 en bajo). Sin embargo, las instrucciones necesarias para evaluar la entrada (cbi, in, andi, cpi, breq) consumen un mínimo físico de 7 ciclos. Por ende, la frecuencia real medida en el osciloscopio se estabiliza alrededor de 1.45 MHz

(16,000,000 / 11 ciclos). La simulación refleja esta compresión máxima en una banda sólida de pulsos continuos.





6 Conclusión

La práctica permitió comprobar que, si bien el lenguaje ensamblador proporciona el control necesario para manipular hardware a nivel de ciclo de reloj, existen limitaciones arquitectónicas infranqueables dictadas por la física del procesador. Se lograron modelar con éxito las frecuencias de 100 kHz, 500 kHz y 1 MHz al compensar los tiempos de las instrucciones en estado lógico bajo. Sin embargo, al solicitar los 2 MHz, se demostró que el *overhead* del muestreo por *polling* (7 ciclos) sobrepasa el presupuesto de tiempo, impidiendo llegar a dicha velocidad. Para futuros desarrollos de altas frecuencias, se recomienda la utilización de los temporizadores hardware (Timers en modo CTC) del AVR en lugar de ciclos de programa.

7 Referencias

- Microchip Technology Inc., *ATmega48A/PA/88A/PA/168A/PA/328/P Datasheet*, Secciones de Arquitectura AVR y Puertos de E/S.

- Apuntes de clase y formato de Actividad I de Microcontroladores.